Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **Информатика и системы управления**
КАФЕДРА **Компьютерные системы и сети (ИУ6)**
НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**
БАКАЛАВРСКАЯ ПРОГРАММА **09.03.01/03 Вычислительные машины, комплексы, системы и сети**

ОТЧЕТ

по лабораторной работе № 2
Вариант 14

«Верификация модуля сдвига и округления мантиссы mant_shifter»

Дисциплина: **Организация ЭВМ и систем**

Студент группы ИУ6-72Б _____ С.Ю. Кондратов
Подпись, дата *(И.О. Фамилия)*

Преподаватель _____ А.Ю. Попов
Подпись, дата *(И.О. Фамилия)*

Оценка _____

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Тестируемое устройство 4
1.1 Интерфейсы устройства 4
1.2 Функциональность 4
2 Тестовое окружение 6
2.1 Интерфейс окружения 6
2.2 Обертка тестируемого устройства 6
2.3 Агенты окружения 7
2.4 Транзакции и драйверы 8
2.5 Мониторы 9
3 Эталонная модель и табло статистики 10
3.1 Проверяемые характеристики 10
3.2 Эталонная модель 11
3.3 Табло статистики 13
3.4 Схема тестового окружения 13
4 Последовательности и тесты 15
4.1 Тестовые сценарии 15
4.2 Рандомизация транзакций 15
4.3 Последовательности 16
4.4 Регрессия 17
4.5 Оценка правильности работы устройства 17
4.6 Результаты CI 20
ЗАКЛЮЧЕНИЕ 24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 25

ВВЕДЕНИЕ

Цель работы – разработать на языке SystemVerilog [3] верификационное окружение для цифрового блока по индивидуальному варианту, дополнить его эталонной моделью, последовательностями и тестами и подтвердить правильность работы блока регрессионным тестированием с полным функциональным покрытием [1].

Задачи работы:

– изучить спецификацию тестируемого устройства и его интерфейсов, создать интерфейс SystemVerilog, обертку устройства и агенты окружения (этап 1);

– составить список проверяемых характеристик, разработать эталонную модель и подключить ее вместе с табло статистики к окружению (этап 2);

– разработать тестовые сценарии, рандомизацию транзакций, последовательности и тесты, собрать регрессию и добиться успешного прохождения CI с полным функциональным и не менее 95 % кодовым покрытием (этап 3).

Задание варианта 14: тестируемое устройство – модуль сдвига и округления мантисс чисел с плавающей точкой mant_shifter [2] с параметром MANT_W = 52, то есть с шириной мантиссы формата двойной точности IEEE 754 [4]. Вся работа выполнена в отдельной ветке iu6_72_kondratov репозитория курса, код каждого этапа отправлен отдельным коммитом с меткой [ci phase N].

Отчет оформлен по ГОСТ 7.32-2017 [6].

1 Тестируемое устройство

1.1 Интерфейсы устройства

Модуль mant_shifter имеет вход тактового сигнала, вход синхронного сброса и два потоковых интерфейса данных с сигналом валидности – входной и выходной (таблица 1). Ширина входа сдвига равна $clog2(MANT_W); при MANT_W = 52 это 6 бит, поэтому величина сдвига принимает значения от 0 до 63, в том числе большие ширины мантиссы.

Таблица 1 – порты модуля mant_shifter при MANT_W = 52

Сигнал	Направление	Ширина	Описание
clk	вход	1	тактовый сигнал
resetn	вход	1	синхронный сброс, активный уровень 0
in_mant	вход	52	входная мантисса
in_shift	вход	6	число разрядов сдвига вправо
in_valid	вход	1	признак валидности входных данных
out_mant	выход	52	сдвинутая и округленная мантисса
out_valid	выход	1	признак валидности выходных данных

Оба интерфейса данных ведут себя как простейшие потоковые интерфейсы с сигнализацией [2]: входные in_mant и in_shift принимаются в такте, где in_valid = 1, а результат выставляется на out_mant вместе с out_valid = 1. Сигнала готовности нет, блок обязан принимать данные в каждом такте. Устройство конвейерное: результат появляется через два такта после входа.

1.2 Функциональность

Блок выполняет сдвиг мантиссы вправо и округление результата – операцию, которая нужна при сложении чисел с плавающей точкой для выравнивания порядков [2, 4]. Перед сдвигом к мантиссе слева дописывается неявная единица, а справа – три дополнительных разряда: G (Guard), R (Round) и S (Sticky). Выпадающие при сдвиге биты попадают сначала в G, затем в R, а бит S равен логическому ИЛИ всех битов, выпавших правее R.

После сдвига разряды G, R, S отбрасываются, а мантисса округляется к ближайшему значению [5]:

– при G = 0 – округление вниз, результат не меняется;

– при G = 1 и R = 1 или S = 1 – округление вверх, к результату прибавляется единица;

– при G = 1, R = 0, S = 0 значение находится ровно посередине, и используется округление к четному: единица прибавляется только при единичном младшем бите результата.

Для варианта 14 отдельно рассматриваются крайние случаи. При нулевом сдвиге мантисса проходит без изменений: неявная единица в разрядную сетку не попадает, а G = R = S = 0. При сдвигах 53–63 мантисса уходит за разрядную сетку целиком, неявная единица оказывается в G (сдвиг 53), в R (сдвиг 54) или в S, и результат равен 0 или 1. Если после сдвига мантисса состоит из одних единиц и округляется вверх, перенос из старшего разряда теряется и результат обнуляется.

2 Тестовое окружение

2.1 Интерфейс окружения

Интерфейс env_if включает все сигналы, выходящие из модуля, сгруппированные по входному и выходному интерфейсам данных (листинг 1). Используется тип logic, так как интерфейс подключается непосредственно к портам устройства и должен отображать значения x и z.

Листинг 1 – файл env_if.sv

```
////////////////////////////////////////////////////////////////////////////////
// Company: Bauman Moscow State Technical University
// Engineer:
//
// Create date: 06/02/2025
// Project Name:
// Module Name:
// Target Devices:
// Tool Versions:
// Description: DUT interface.
////////////////////////////////////////////////////////////////////////////////

interface env_if ();
  logic        clk;
  logic        resetn;

  // Входной интерфейс. MANT_W = 52, ширина сдвига $clog2(52) = 6 бит
  logic [51:0] in_mant;
  logic [5:0]  in_shift;
  logic        in_valid;

  // Выходной интерфейс
  logic [51:0] out_mant;
  logic        out_valid;

endinterface : env_if
```

2.2 Обертка тестируемого устройства

В обертке tb_dut_wrapper объявлен модуль mant_shifter с параметром MANT_W = 52, его порты подключены к сигналам интерфейса (листинг 2). В Makefile переменной DUT присвоено имя mant_shifter.

Листинг 2 – файл tb_dut_wrapper.sv

```
//////////////////////////////////////////////////////////////////////////////////
// Company: Bauman Moscow State Technical University
// Engineer:
//
// Create date: 11/02/2025
// Project Name:
// Module Name:
// Target Devices:
// Tool Versions:
// Description: DUT wrapper.
//////////////////////////////////////////////////////////////////////////////////

module tb_dut_wrapper (env_if env_intf);

  // DUT по варианту: mant_shifter, MANT_W = 52
  mant_shifter # (
    .MANT_W(52)
  )
  mant_shifter_inst (
    .clk      (env_intf.clk),
    .resetn   (env_intf.resetn),
    .in_mant  (env_intf.in_mant),
    .in_shift (env_intf.in_shift),
    .in_valid (env_intf.in_valid),
    .out_mant (env_intf.out_mant),
    .out_valid(env_intf.out_valid)
  );

endmodule : tb_dut_wrapper
```

2.3 Агенты окружения

Для каждого логически самостоятельного интерфейса устройства используется свой агент (таблица 2). Агенты тактового сигнала и сброса предоставлены в репозитории. Входной и выходной интерфейсы данных ведут себя одинаково, но имеют разный набор сигналов (на входе есть величина сдвига), поэтому для них созданы два разных агента: входной работает в активном режиме, выходной – в пассивном.

Таблица 2 – агенты тестового окружения

Агент	Режим	Сигналы	Назначение
clk_agent	активный	clk	генерация тактового сигнала с периодом 40 нс
resetn_agent	активный	resetn	выставление сброса и формирование транзакций начала и конца сброса
mant_in_agent	активный	in_mant, in_shift, in_valid	подача входных мантисс и сбор входных транзакций
mant_out_agent	пассивный	out_mant, out_valid	сбор выходных транзакций устройства

Каждый агент собран из транзакции, драйвера и монитора и унаследован от базового класса env_base_agent, секвенсор используется базовый. Классы агента собраны в собственный пакет (mant_in_agent_pkg, mant_out_agent_pkg), пакеты подключены к env_pkg, а каталоги агентов добавлены в files_tb.mk. В классе env агенты создаются в методе build(), получают ссылку на событие stop_event, режим is_active и виртуальный интерфейс и запускаются в методе run().

2.4 Транзакции и драйверы

Транзакция mant_in_tran содержит рандомизируемые поля mant (52 бита) и shift (6 бит) и служебное поле back_to_back, которое не относится к данным, а подсказывает драйверу, как подать транзакцию. Транзакция mant_out_tran содержит одно поле mant – выходную мантиссу. В обеих транзакциях переопределены методы compare() (сравнение полей данных), copy() (копирование полей из другой транзакции того же класса) и print() (вывод полей в шестнадцатеричном виде).

Драйвер mant_in_driver в методе init_signals() обнуляет in_mant и in_shift и снимает in_valid, так что после сброса на входе нет валидных данных. Метод drive_tran() по переднему фронту выставляет мантиссу, сдвиг и in_valid = 1. Если поле back_to_back сброшено, драйвер ждет следующий фронт, на котором устройство принимает данные, и возвращает входы в исходное состояние – получается разреженный поток. Если поле установлено, in_valid

не снимается и следующая транзакция выставляется уже на следующем такте – получается плотный поток, в котором данные идут каждый такт.

У выходного интерфейса нет сигнала готовности, поэтому драйвер mant_out_driver ничего не выставляет: метод drive_passive() не переопределен, а drive_tran() только выводит предупреждение о том, что агент должен работать в пассивном режиме.

2.5 Мониторы

Мониторы работают по заднему фронту тактового сигнала, когда сигналы уже установились, и не конфликтуют с драйвером, который работает по переднему фронту. Монитор mant_in_monitor в методе check_validity() считает такт валидным при in_valid = 1, а метод monitor_tran() собирает in_mant и in_shift в транзакцию mant_in_tran и отправляет ее в mon2out_mbx. Монитор mant_out_monitor аналогично собирает out_mant при out_valid = 1. Внутренних переменных у мониторов нет, поэтому метод reset() не переопределен; во время сброса базовый класс монитора транзакции не собирает.

3 Эталонная модель и табло статистики

3.1 Проверяемые характеристики

По разделам спецификации «поведение интерфейсов» и «функциональность» [2] составлен список проверяемых характеристик (таблицы 3 и 4), он же занесен в план верификации.

Таблица 3 – проверяемые характеристики интерфейсов

Интерфейс	Предмет проверки	Способ проверки	Важность
Тактовый сигнал	переключение блока по фронту clk, отсутствие «залипаний»	временная диаграмма	крит.
Сброс	синхронный сброс по resetn = 0	временная диаграмма, нет выходных транзакций во время сброса	крит.
	сброс во время работы: данные в конвейере теряются, после сброса блок работает	временная диаграмма, эталонная модель	важн.
Входной	прием данных только при in_valid = 1	временная диаграмма, эталонная модель	крит.
	прием каждый такт и с паузами	эталонная модель, функциональное покрытие	крит.
Выходной	выдача out_mant вместе с out_valid через 2 такта	временная диаграмма, эталонная модель	крит.
	нет out_valid без входных данных	эталонная модель (нет лишних транзакций)	крит.

Таблица 4 – проверяемые функции

Функция	Предмет проверки	Важность
Сдвиг вправо	мантисса с неявной единицей сдвигается на in_shift разрядов при сдвигах 1–52	крит.
Нулевой сдвиг	мантисса проходит без изменений, неявная единица в результат не попадает	важн.
Сдвиг больше ширины мантиссы	при сдвигах 53–63 результат 0 или 1 в зависимости от G, R, S	важн.
Округление вниз	G = 0: результат не меняется	крит.
Округление вверх	G = 1 и R = 1 или S = 1: к результату прибавляется 1	крит.
Округление к четному	G = 1, R = 0, S = 0: единица прибавляется только при младшем бите 1	крит.
Формирование бита S	S = 1, если выпала хотя бы одна единица правее R, при любом сдвиге	крит.
Переполнение при округлении	все единицы после сдвига и округление вверх дают 0	средне
Конвейерная работа	данные, поданные каждый такт, обрабатываются без потерь и по порядку	крит.

3.2 Эталонная модель

Эталонная модель mant_shifter_ref унаследована от env_base_ref с параметром resetn_transaction. У нее два входных порта – in_mbx (транзакции входного агента) и rst_mbx (транзакции агента сброса, порт базового класса) – и один выходной порт out_mbx, который создается в конструкторе. Входные порты подключаются к mon2out_mbx агентов в методе connect() окружения.

Результат вычисляется функцией shift_round() по правилам спецификации, а не повторением схемы устройства (листинг 3). В RTL к мантиссе дописываются 63 нулевых разряда, и S получается как ИЛИ их хвоста, а модель явно выделяет G, R и S из выпавших битов. Совпадение двух независимых реализаций повышает доверие к результату проверки. Сдвиг копируется в знаковую переменную sh: при беззнаковом сдвиге выражение sh − 2 для сдвига 1 переполнялось бы и давало неверный S.

Листинг 3 – функция shift_round() эталонной модели (файл mant_shifter_ref.sv)

```
function bit [51:0] mant_shifter_ref::shift_round(bit [51:0] mant, bit
    [5:0] shift);
  bit [52:0] full;   // мантисса с неявной единицей
  bit [51:0] res;
  bit        g, r, s;
  int        sh;     // знаковая копия сдвига, иначе sh - 2 при сдвиге 1
     уйдет в огромное число

  full = {1'b1, mant};
  sh   = shift;

  // При сдвиге 0 мантисса не меняется: неявная единица в разрядную сетку
     не попадает
  res = full >> sh;

  // Выпавшие биты: G - первый, R - второй, S - ИЛИ всех остальных
  g = 0;
  r = 0;
  s = 0;
  for (int i = 0; i <= 52; i++) begin
    if (i == sh - 1)      g = full[i];
    else if (i == sh - 2) r = full[i];
    else if (i < sh - 2)  s |= full[i];
  end

  // Округление к ближайшему, ровно посередине - к четному
  if (g && (r || s || res[0]))
    res++;

  return res;
endfunction : shift_round
```

Основной цикл main_loop() берет транзакцию из in_mbx, пропускает ее при активном сбросе и вычисляет результат. Поскольку устройство конвейерное и следующая транзакция может прийти, пока предыдущая еще в конвейере, отправка результата запускается отдельным потоком (fork … join_none) задачей send_out() (листинг 4). Задача ждет два такта – задержку устройства – и выбрасывает результат, если за это время resetn опустился: сброс очищает конвейер устройства, и этого результата на выходе не будет. Для отсчета тактов модели передан виртуальный интерфейс, через который она ничего не выставляет. Внутреннего состояния у модели нет, поэтому метод process_reset() пустой.

Листинг 4 – задача send_out() эталонной модели (файл mant_shifter_ref.sv)

```
task mant_shifter_ref::send_out(mant_out_tran out_tran);
  // Вход монитор видит по negedge, результат на выходе появляется через 2
     такта.
  // Если за эти 2 такта resetn упал, DUT очистил конвейер и результата не
     будет.
  // Смотрим прямо на resetn: на negedge он стабилен, а флаг reset может
     выставиться
  // в этот же момент и мы его не увидим.
  repeat (2) begin
    @ (negedge vif.clk);
    if (!vif.resetn) return;
  end

  out_mbx.put(out_tran);
endtask : send_out
```

Такой подход выбран после проверки базового табло статистики: ожидание новых элементов в нем фактически срабатывает только в конце теста, поэтому очищать выходную очередь модели при сбросе нельзя – вместе с потерянными результатами были бы выброшены и результаты, уже собранные с выхода устройства, но еще не сравненные.

3.3 Табло статистики

К окружению подключен базовый класс env_scoreboard с параметром mant_out_tran. Порт dut_mbx подключен к монитору выходного агента, порт ref_mbx – к выходу эталонной модели. Табло сравнивает транзакции попарно методом compare(), а метод check() выводит статистику: число транзакций, сравнений, ошибок сравнения и «сирот» – транзакций без пары. Тест считается пройденным, если ошибок и сирот нет; метод check() класса base_test возвращает результат env0.scb.check().

3.4 Схема тестового окружения

Итоговая схема окружения приведена на рисунке 1. Агенты тактового сигнала и сброса и входной агент связаны с интерфейсом в обе стороны: драйвер выставляет сигналы, монитор их же считывает. Выходной агент только считывает сигналы. Транзакции между компонентами передаются через mailbox в одну сторону.

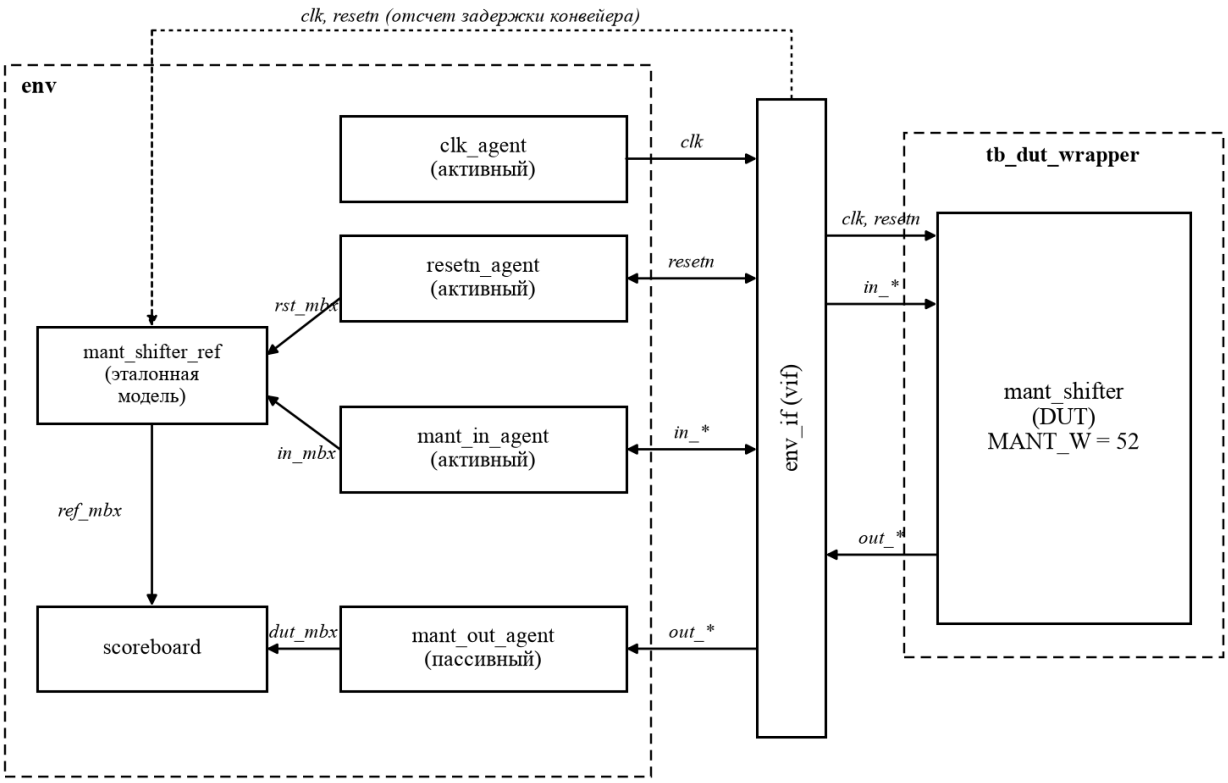


Рисунок 1 – схема тестового окружения

4 Последовательности и тесты

4.1 Тестовые сценарии

Набор тестов составлен так, чтобы покрыть все функции и характеристики из таблиц 3 и 4 (таблица 5). Каждый тест унаследован от base_test, зарегистрирован в фабрике макросом factory_register и переопределяет метод run_main(); все тесты, кроме базового, начинают с паузы в 10 тактов после первичного сброса.

Таблица 5 – тестовые сценарии

Тест	Что проверяет	Ход действий
`base`	сборку окружения, тактовый сигнал и первичный сброс	запуск тактового сигнала и сброса, ожидание 100 тактов
`shift`	сдвиг на все значения 0–63, округление вверх и вниз, разреженный поток	для каждого сдвига 0–63 – от 4 до 8 случайных мантисс с паузами, затем 5–10 тактов ожидания
`round`	все случаи округления, бит S, переполнение, сдвиги 0, 53, 63	5 раз подряд – по одной транзакции на каждый из 10 крайних случаев
`stream`	конвейерную работу, плотный и разреженный потоки	10 раз – пачка из 10–50 транзакций каждый такт, затем 10–50 транзакций с паузами
`reset`	горячий сброс и работу после него	4 раза – плотный поток из 20–30 транзакций одновременно со сбросом, затем такой же поток после сброса

4.2 Рандомизация транзакций

В транзакции mant_in_tran рандомизируются поля данных mant и shift. Мантисса не ограничивается: при сдвиге на входе допустимо любое из 2^52 значений, а случайные биты дают оба исхода округления и переключают все разряды устройства. Поле shift покрывает весь диапазон 0–63, но значимые для устройства области неравновероятны: нулевой сдвиг – одно значение, сдвиг больше ширины мантиссы – 11 значений. Поэтому для shift задано весовое ограничение (листинг 5).

Листинг 5 – ограничение на величину сдвига (файл mant_in_tran.sv)

```
constraint shift_ctr {
  shift dist {0 :/ 1, [1:52] :/ 8, [53:63] :/ 1};
}
```

Оператор :/ делит вес на весь диапазон, поэтому три области – без сдвига, сдвиг внутри мантиссы и сдвиг за ее пределы – выпадают в

соотношении 1 : 8 : 1 независимо от числа значений в каждой. Ограничение не конфликтует с дополнительными inline-ограничениями последовательностей, так как охватывает все значения поля. Поле back_to_back не рандомизируется – его задает последовательность.

4.3 Последовательности

Последовательность mant_seq формирует пачку случайных транзакций. Ее настройки: границы числа транзакций min_tran_num и max_tran_num, диапазон сдвига min_shift…max_shift и режим back_to_back. Число транзакций tran_num рандомизируется перед запуском в заданных границах, а каждая транзакция рандомизируется конструкцией randomize() with с ограничением shift inside {[min_shift:max_shift]}. В режиме back_to_back флаг ставится всем транзакциям, кроме последней, чтобы после пачки сигнал in_valid обязательно снимался. Одна гибкая последовательность используется во всех тестах с потоком данных: тест сдвига фиксирует диапазон сдвига, а тесты потока и сброса включают плотный режим.

Последовательность mant_round_seq отправляет по одной транзакции на каждый из 10 крайних случаев округления. Случай задается inline-ограничением только на значимые биты, остальные биты мантиссы остаются случайными. Для сдвига 5 разряд G – это mant[4], R – mant[3], S – ИЛИ mant[2:0], а младший бит результата – mant[5], поэтому, например, округление к четному при младшем бите 1 задается ограничением mant[5:0] == 6'b110000. Кроме того, проверяются переполнение (все единицы, сдвиг 1), нулевой сдвиг, сдвиг 53 с нулевой и ненулевой мантиссой и максимальный сдвиг 63.

Виртуальные последовательности не понадобились: сценарии тестов сводятся к последовательному или параллельному запуску обычных последовательностей прямо из run_main(). В тесте сброса последовательность данных и последовательность сброса запускаются параллельно в блоке fork … join, без обращения к виртуальному интерфейсу.

4.4 Регрессия

Все тесты включены в пакет test_pkg и добавлены в список REGRESSION_TESTS в Makefile: base, shift, round, stream, reset. По команде make regression они запускаются по очереди, затем выводится статистика и объединяется база покрытия. В скрипт add_signals.tcl добавлены сигналы in_mant, in_shift, in_valid, out_mant и out_valid для просмотра временных диаграмм в графическом режиме.

4.5 Оценка правильности работы устройства

Значения out_mant для всех функций из таблицы 4 проверяются эталонной моделью: во всех тестах регрессии ошибок сравнения и сирот нет. Поведение интерфейсов проверено по временным диаграммам, полученным моделированием окружения в Verilator [7] и снятым в GTKWave [8]. Кроме сигналов из add_signals.tcl, на них выведены сигналы первой стадии конвейера устройства: stage1_valid, stage1_mant, stage1_G, stage1_R, stage1_S и round_up.

На рисунке 2 показан разреженный поток в тесте shift при нулевом сдвиге: каждая транзакция держит in_valid один такт, между транзакциями пауза в один такт. Результат выставляется с out_valid через два такта и совпадает со входной мантиссой, что подтверждает и нулевой сдвиг без неявной единицы, и задержку конвейера.

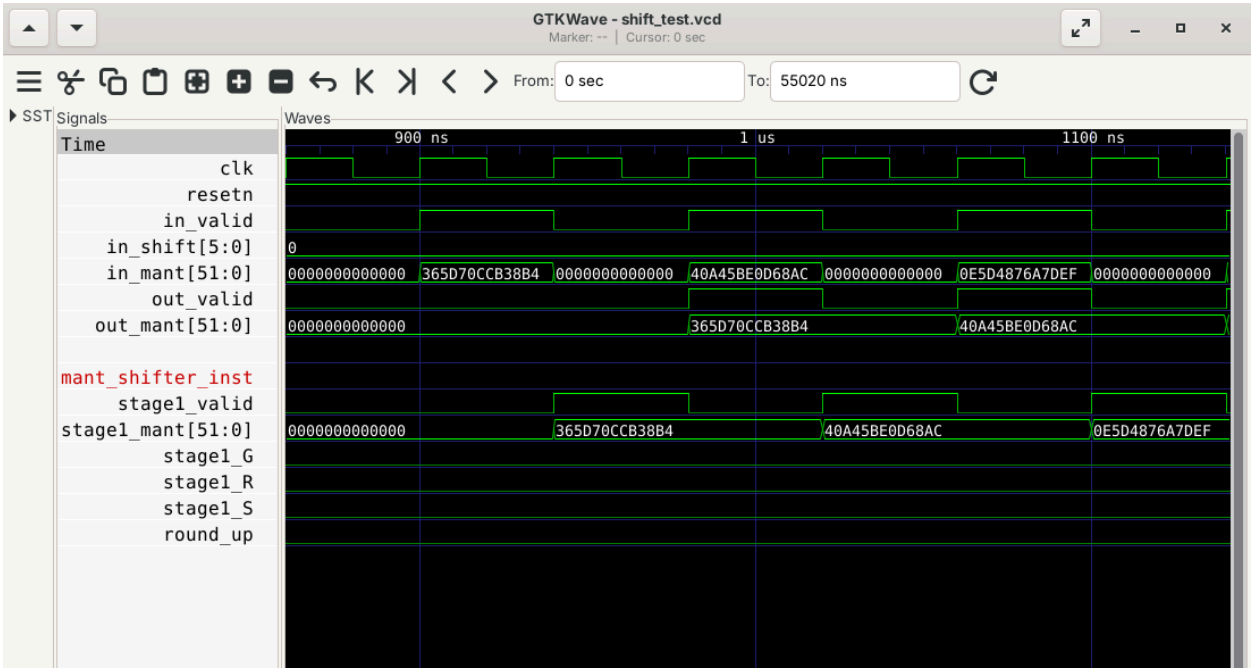


Рисунок 2 – разреженный поток и нулевой сдвиг (тест shift)

На рисунке 3 показан плотный поток в тесте stream: in_valid не снимается, и новые данные поступают каждый такт. Через два такта out_valid устанавливается и держится, результаты идут подряд без пропусков. Например, мантисса 69D9613B464F1 со сдвигом 7 дает 02D3B2C2768CA: на первой стадии конвейера G = 1, R = 1, S = 1, поэтому round_up = 1 и результат округлен вверх.

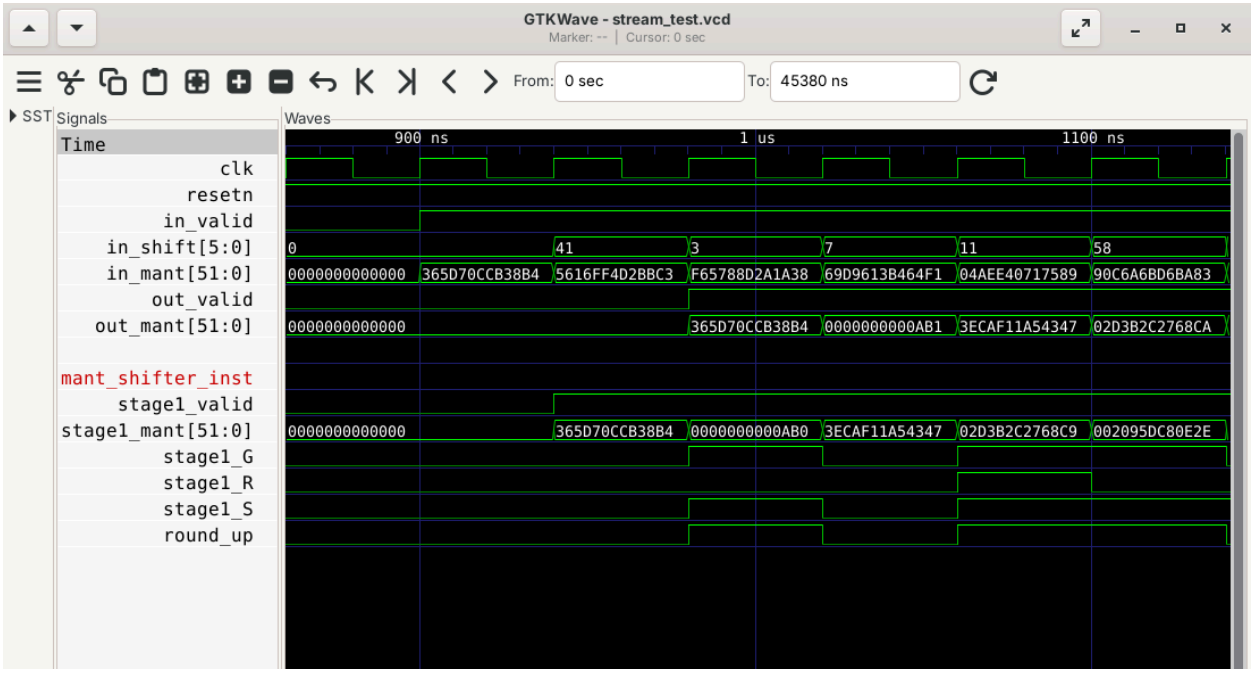


Рисунок 3 – плотный поток (тест stream)

На рисунке 4 показан горячий сброс в тесте reset. Сигнал resetn опускается посреди плотного потока, и уже на следующем фронте out_valid снимается: транзакции, находившиеся в конвейере, теряются. Входные данные, которые драйвер еще выставляет в первые такты сброса, устройство игнорирует, а во время сброса выходных транзакций нет. Эталонная модель предсказывает эти потери, поэтому табло статистики не находит ни ошибок, ни сирот.

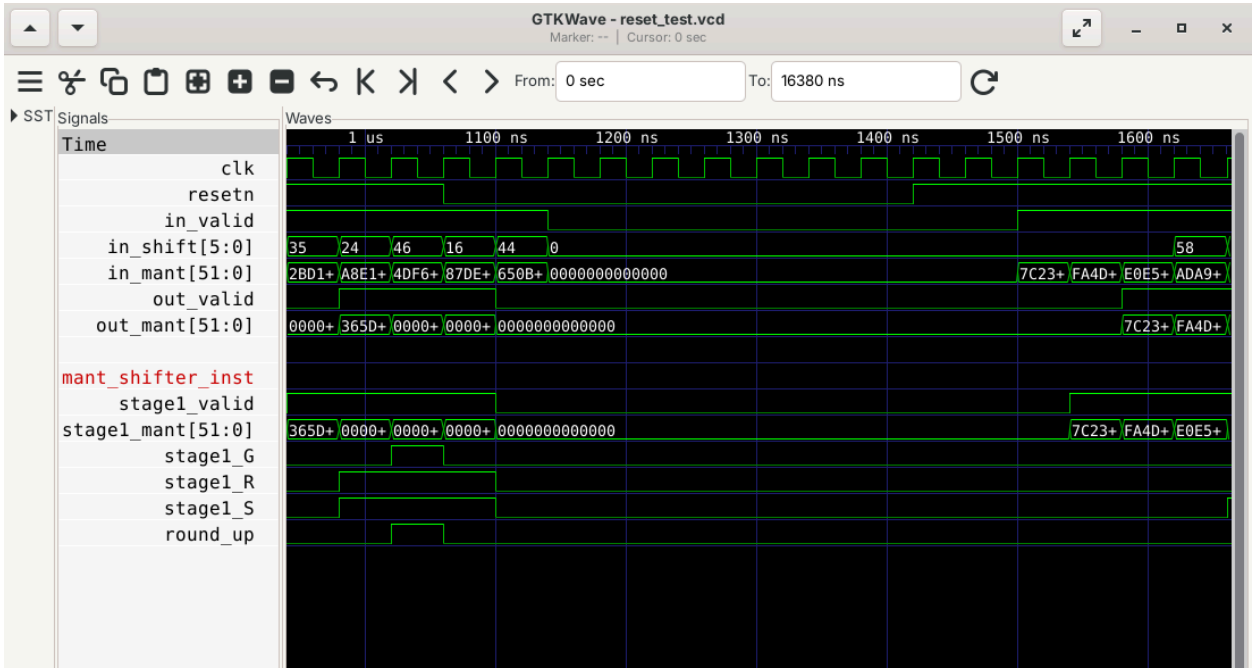


Рисунок 4 – сброс во время плотного потока (тест reset)

Результаты первого прохода теста round по всем крайним случаям округления приведены в таблице 6. Ожидаемое значение рассчитано по правилам спецификации, полученное взято с выхода устройства; во всех 50 транзакциях теста они совпали.

Таблица 6 – крайние случаи округления (тест round)

Случай	in_mant	Сдвиг	GRS	Ожидаемый out_mant	Полученный out_mant
вниз (G = 0)	FBE68B8B35B2B	5	011	0FDF345C59AD9	0FDF345C59AD9
вверх по R	E6161215CE9FA	5	111	0F30B090AE750	0F30B090AE750
вверх по S	CFBADD66D4FF5	5	101	0E7DD6EB36A80	0E7DD6EB36A80
к четному, LSB = 0	64C38786C41D0	5	100	0B261C3C3620E	0B261C3C3620E
к четному, LSB = 1	5DF0AFDC160B0	5	100	0AEF857EE0B06	0AEF857EE0B06
переполнение	FFFFFFFFFFFF	1	100	0000000000000	0000000000000
нулевой сдвиг	FFFFFFFFFFFF	0	000	FFFFFFFFFFFF	FFFFFFFFFFFF
сдвиг 53, ровно	0000000000000	53	100	0000000000000	0000000000000
сдвиг 53, вверх	CFDE4A6272F8D	53	111	0000000000001	0000000000001
сдвиг 63	9AE0FAC713C45	63	001	0000000000000	0000000000000

Функциональное покрытие задано моделью покрытия в RTL устройства (covergroup mant_shifter_cg). Ее бины и тесты, которые их закрывают, приведены в таблице 7.

Таблица 7 – бины функционального покрытия

Бин	Что означает	Закрывающие тесты
ZERO_SHIFT	сдвиг 0	shift, round
SHIFT_LESSER_THAN_MANTISSA_WIDTH	сдвиг 1–52	все тесты с данными
SHIFT_GREATER_THAN_MANTISSA_WIDTH	сдвиг 53–63	shift, round, stream
INPUT_VALUES_SPARSE_STREAM	разреженный поток	shift, round, stream
INPUT_VALUES_TIGHT_STREAM	три транзакции подряд	stream, reset
AFTER_GRS_ROUND_UP	округление вверх	все тесты с данными
AFTER_GRS_ROUND_DOWN	округление вниз	все тесты с данными

4.6 Результаты CI

Код каждого этапа отправлен в ветку iu6_72_kondratov отдельным коммитом с меткой [ci phase N], и пайплайн каждого этапа завершился успешно. На этапах 1 и 2 базовый тест со встроенными проверками этапа прошел без ошибок (рисунки 5 и 6). На этапе 3 регрессия из пяти тестов прошла полностью: 5 PASSED, 0 FAILED (рисунок 7). Кодовое покрытие устройства составило 97,18 % при требуемых 95 % (рисунок 8), функциональное покрытие – 100 % (рисунок 9).

ИУ6 - Секция аппаратное обеспечение / ЛБ Верификация / Jobs / #76167

Passed build-phase-1

Search visible log output

```
134 # do script/run_all.tcl
135 # Test base_test is found in plusargs, added to queue.
136 # [base_test] started.
137 # 0 [base_test.env0.clk_sig_agent.monitor] started.
138 # 0 [base_test.env0.resetn_sig_agent.driver] started.
139 # 0 [base_test.env0.resetn_sig_agent.monitor] started.
140 # 0 [base_test.env0.in_agent.driver] started.
141 # 0 [base_test.env0.in_agent.monitor] started.
142 # 0 [base_test.env0.out_agent.driver] started.
143 # 0 [base_test.env0.out_agent.monitor] started.
144 # 0 [base_test.env0.clk_sig_agent.driver] recieved new clock transaction.
145 # 20000 [base_test.env0.resetn_sig_agent.driver] recieved new transaction.
146 # 60000 [base_test.env0.resetn_sig_agent.driver] is driving the transaction.
147 # 4620000 [base_test.env0.clk_sig_agent.monitor] stopped.
148 # 4620000 [base_test.env0.resetn_sig_agent.monitor] stopped.
149 # 4620000 [base_test.env0.in_agent.monitor] stopped.
150 # 4620000 [base_test.env0.out_agent.monitor] stopped.
151 # [base_test] WARNING: check method should be re-written or implemented in child class!
152 # ** Note: $stop    : /home/gitlab-runner/builds/sMvyx6TfH/0/iu6-hardware-section/verification_lab/unit_verif/testbench/tb_
    top.sv(41)
153 #    Time: 4820 ns  Iteration: 1  Instance: /tb_top
154 # Break in Module tb_top at /home/gitlab-runner/builds/sMvyx6TfH/0/iu6-hardware-section/verification_lab/unit_verif/testben
    ch/tb_top.sv line 41
155 # Stopped at /home/gitlab-runner/builds/sMvyx6TfH/0/iu6-hardware-section/verification_lab/unit_verif/testbench/tb_top.sv li
    ne 41
156 # Saving coverage database on exit...
157 # End time: 11:57:43 on Oct 26,2026, Elapsed time: 0:00:03
158 # Errors: 0, Warnings: 0
159 +------------------------------------------------------------------------------+
160 |                                                                              |
161 |                               TEST PASSED                                    |
162 |                                                                              |
163 +------------------------------------------------------------------------------+
164   SEED : 40665
165 Cleaning up project directory and file based variables                    00:00
166 Job succeeded
```

Рисунок 5 – лог джобы build-phase-1 (этап 1)

Passed build-phase-2

Search visible log output

```
138 # 0 [base_test.env0.resetn_sig_agent.driver] started.
139 # 0 [base_test.env0.resetn_sig_agent.monitor] started.
140 # 0 [base_test.env0.in_agent.driver] started.
141 # 0 [base_test.env0.in_agent.monitor] started.
142 # 0 [base_test.env0.out_agent.driver] started.
143 # 0 [base_test.env0.out_agent.monitor] started.
144 # 0 [base_test.env0.reference] started.
145 # 0 [base_test.env0.scb] started.
146 # 0 [base_test.env0.clk_sig_agent.driver] recieved new clock transaction.
147 # 20000 [base_test.env0.resetn_sig_agent.driver] recieved new transaction.
148 # 60000 [base_test.env0.resetn_sig_agent.driver] is driving the transaction.
149 # 4660000 [base_test.env0.clk_sig_agent.monitor] stopped.
150 # 4660000 [base_test.env0.resetn_sig_agent.monitor] stopped.
151 # 4660000 [base_test.env0.in_agent.monitor] stopped.
152 # 4660000 [base_test.env0.out_agent.monitor] stopped.
153 # 4660000 [base_test.env0.scb] stopped.
154 # 4660000 [base_test.env0.reference] stopped.
155 # [base_test] WARNING: check method should be re-written or implemented in child class!
156 # ** Note: $stop    : /home/gitlab-runner/builds/sMvyx6TfH/0/iu6-hardware-section/verification_lab/unit_verif/testbench/tb_
    top.sv(41)
157 #    Time: 4860 ns  Iteration: 1  Instance: /tb_top
158 # Break in Module tb_top at /home/gitlab-runner/builds/sMvyx6TfH/0/iu6-hardware-section/verification_lab/unit_verif/testben
    ch/tb_top.sv line 41
159 # Stopped at /home/gitlab-runner/builds/sMvyx6TfH/0/iu6-hardware-section/verification_lab/unit_verif/testbench/tb_top.sv li
    ne 41
160 # Saving coverage database on exit...
161 # End time: 12:33:59 on Oct 26,2026, Elapsed time: 0:00:03
162 # Errors: 0, Warnings: 0
163 +------------------------------------------------------------------------------+
164 |                                                                              |
165 |                               TEST PASSED                                    |
166 |                                                                              |
167 +------------------------------------------------------------------------------+
168   SEED : 52097
169 Cleaning up project directory and file based variables                    00:00
170 Job succeeded
```

Рисунок 6 – лог джобы build-phase-2 (этап 2)

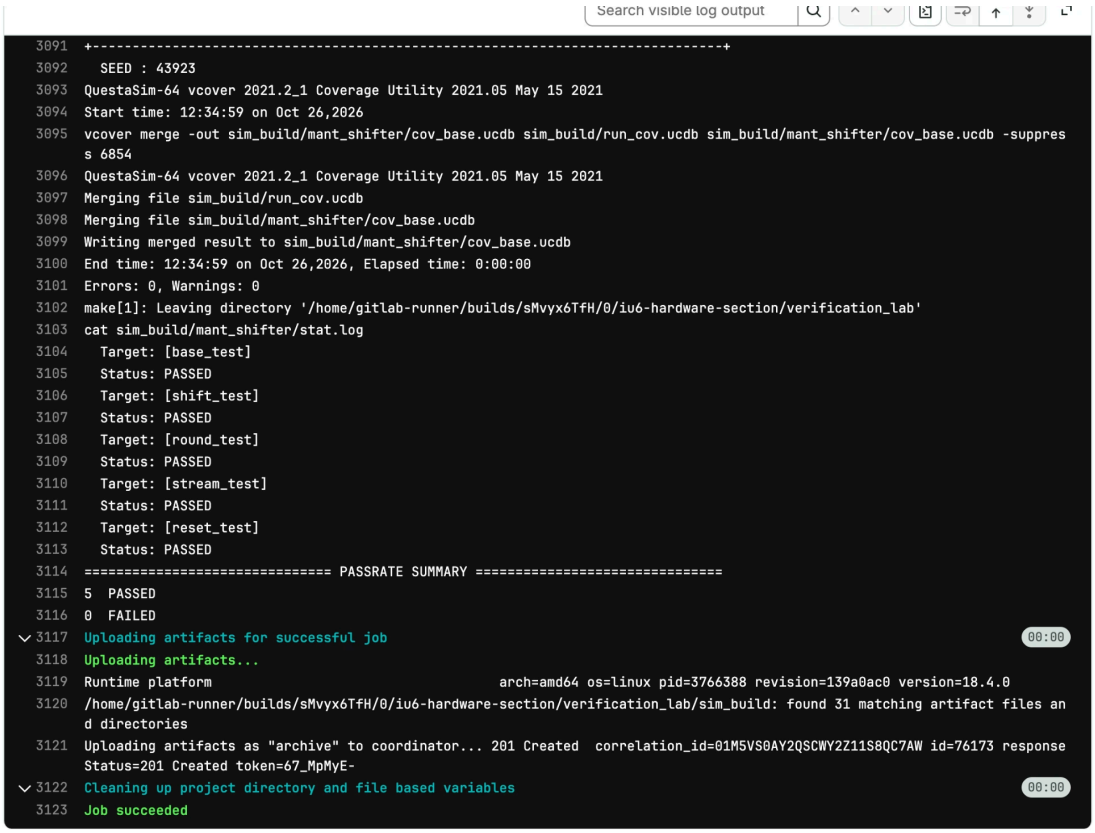


Рисунок 7 – результаты регрессии, джоба test-phase-3 (этап 3)

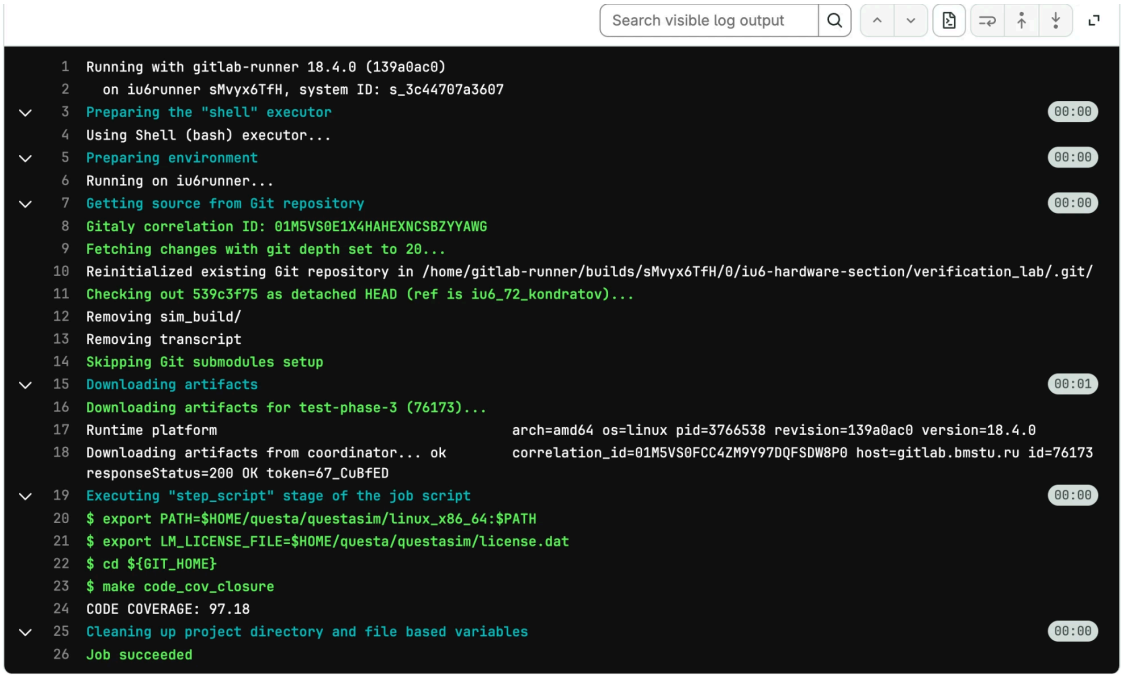


Рисунок 8 – кодовое покрытие, джоба code-coverage (этап 3)

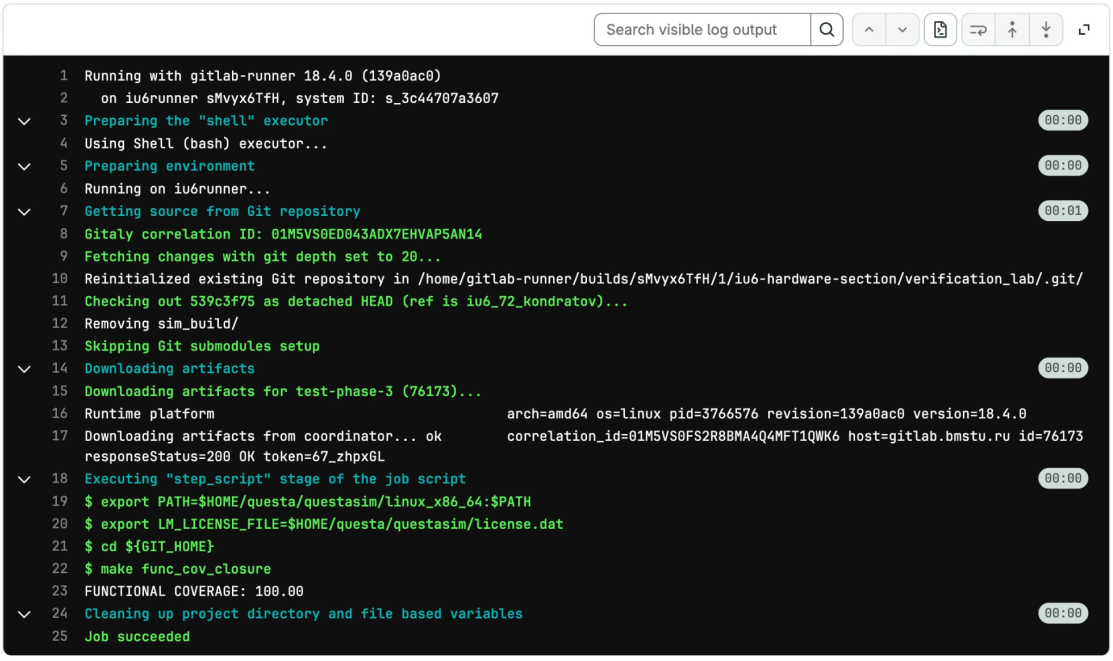


Рисунок 9 – функциональное покрытие, джоба func-coverage (этап 3)

ЗАКЛЮЧЕНИЕ

Для модуля сдвига и округления мантисс mant_shifter с MANT_W = 52 разработано верификационное окружение на SystemVerilog: интерфейс, обертка устройства, активный входной и пассивный выходной агенты, эталонная модель и табло статистики. Эталонная модель вычисляет результат по правилам спецификации через явные разряды G, R, S и учитывает двухтактную задержку конвейера, поэтому правильно предсказывает потерю данных при горячем сбросе.

Разработаны пять тестов, две последовательности и весовое ограничение на величину сдвига. Регрессия проходит без ошибок сравнения, кодовое покрытие устройства составило 97,18 %, функциональное – 100 %, пайплайны CI всех трех этапов завершились успешно. Расхождений поведения устройства со спецификацией не обнаружено: сдвиг, все варианты округления, крайние случаи сдвигов 0 и 53–63, переполнение при округлении, конвейерная работа и сброс соответствуют ожидаемым.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторные работы и ДЗ по верификации. Этапы № 1 – № 3: методические указания [Электронный ресурс] // Репозиторий verification_lab, МГТУ им. Н.Э. Баумана, кафедра ИУ6. – URL: https://gitlab.bmstu.ru/iu6-hardware-section/verification_lab (дата обращения: 26.10.2026).

2. Спецификация модуля сдвига и округления мантисс чисел с плавающей точкой mant_shifter [Электронный ресурс] // Репозиторий verification_lab. – URL: https://gitlab.bmstu.ru/iu6-hardware-section/verification_lab (дата обращения: 26.10.2026).

3. IEEE Std 1800-2017. IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language. – New York: IEEE, 2018. – 1315 p.

4. IEEE Std 754-2019. IEEE Standard for Floating-Point Arithmetic. – New York: IEEE, 2019. – 84 p.

5. Floating-point numbers and rounding [Электронный ресурс]. – URL: https://drilian.com/posts/2023.01.10-floating-point-numbers-and-rounding/ (дата обращения: 26.10.2026).

6. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017. – 27 с.

7. Verilator [Электронный ресурс]. – URL: https://verilator.org (дата обращения: 26.10.2026).

8. GTKWave [Электронный ресурс] : программа просмотра временных диаграмм. – URL: https://gtkwave.sourceforge.net (дата обращения: 26.10.2026).